

DIGITAL FORENSICS LABORATORY · LAB 05

Network Traffic Forensics

Working paper — packets, streams, exports, and covert channels in Wireshark

Name	Student ID (NIM)	Class / Group
------	------------------	---------------

Date	Instructor / TA	Session grade
------	-----------------	---------------

1. Lab Overview

This working paper accompanies Lab 05 of the digital forensics series. The same attacker from the Lab 04 breach is back - this time recorded on the wire. You will orient inside a packet capture in minutes using protocol hierarchy and conversation statistics, read entire conversations with Follow TCP Stream, recover transferred files with Export Objects, and decode layered payloads including an XOR-encrypted covert channel. Complete the pre-lab before the session, check off each step as you perform it, cite packet and stream numbers in every answer, and close with a reflection.

1.1 Learning objectives

- Explain how IP addresses, protocols, and ports identify a network conversation.
- Capture traffic with Wireshark/tcpdump and distinguish .pcap from .pcapng.
- Orient in a capture: protocol hierarchy, conversations, capinfos metadata.
- Write display filters that isolate protocols, ports, hosts, and commands.
- Reassemble conversations with Follow TCP Stream and cite them as evidence.
- Recover transferred files with Export Objects (and hash them as evidence).
- Recognize on the wire: brute force, credential leaks, reverse shells, covert channels.
- Decode layered payloads: URL encoding, hex, base64, and repeating-key XOR.

1.2 Environment and files

Environment: Kali Linux (or any machine with Wireshark; tshark ships with it). Copy every capture to your working directory first and hash it - captures are evidence, and full-payload captures are confidential by nature.

Table 1.1 — Lab files

File	Purpose
files/capture.pcapng	Guided drill — the teaching capture from the README (upstream)
files/challenge.pcapng	Challenge A — 2,938 packets of the recorded breach (upstream)
files/backdoor.py	Challenge A — the script the attacker downloaded (read it!)
files/office_hours.pcap	Challenge B (new) — a quiet intranet capture with a secret
files/ping_puzzle.pcap	Challenge C (new) — 112 packets of ICMP with something to say

1.3 What to submit

- Section 1.4 pre-lab, answered before the session (worth 10 points).
- This working paper as PDF, named NIM_Name_Lab05.pdf.
- Your evidence folder NIM_Lab05/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- Answers that cite exact packet numbers, stream numbers, or filters - not vague descriptions.

1.4 Pre-Lab Check

Answer these before the session, using the lab README and Section 2 of this paper as references. One or two sentences per question are enough - precision beats length.

P1. What three things (address, protocol, port) identify one side of a network conversation, and which common ports belong to FTP, SSH, HTTP, and MySQL?

Answer:

Two sentences maximum.

P2. What is the difference between .pcap and .pcapng, and which tool produces each by default?

Answer:

Two sentences maximum.

P3. What does Follow TCP Stream show that the packet list does not?

Answer:

Two sentences maximum.

P4. Why are FTP, HTTP, and Telnet the forensic analyst's favorite protocols, while HTTPS is not?

Answer:

Two sentences maximum.

P5. What is a covert channel, and give one example of hiding data inside a 'normal' protocol.

Answer:

Two sentences maximum.

2. Theory Primer

2.1 Conversations: IP, protocol, port

Every packet belongs to a conversation between (source IP : source port) and (destination IP : destination port) using a protocol. Servers listen on well-known ports (FTP 21, SSH 22, HTTP 80, HTTPS 443, MySQL 3306); clients pick random high ports. An investigator reads these coordinates like an address: who talked, to which service, and - via timestamps - in what order. Ports that fit no known service (4444, 5555) are instant suspicion.

2.2 The Wireshark workflow

Orient before you scroll. Statistics -> Protocol Hierarchy shows the protocol mix in one screen; Conversations ranks talkers by packets and bytes; capinfos prints the capture window and hash. Then filter (http, ftp, tcp.port == 5555, ip.addr == x) to carve out what matters. Then Follow TCP Stream to read whole conversations with one side colored red and the other blue. Finally Export Objects (File menu) recovers files that crossed HTTP/SMB/FTP-DATA without manual reassembly. Filter strings can be built by right-clicking any field in the details pane.

2.3 Attacks as they appear on the wire

Table 2.1 — Wire signatures

Behavior	What you see
Brute force	many short streams to one service port; repeated USER/PASS with failure replies (FTP 530)
Credential leak	plaintext logins readable in Follow Stream (FTP, HTTP forms, Telnet)
Web exploitation	POST requests with encoded command parameters; error-shaped responses
Reverse shell	traffic to an odd high port; one side types commands, the other echoes output
Data exfiltration	large outbound transfers, exported files, unexpected TLS to file hosts
Covert channel	protocol-correct traffic whose payload is not what that protocol normally carries

2.4 Decoding layered payloads

URL encoding (%XX, + for space) wraps most HTTP payloads - decode once, and if % signs remain, decode again. Hex strings decode with `bytes.fromhex`; base64 with `base64 -d`. A repeating-key XOR hides the backdoor channel in this lab: read the key from the script the attacker downloaded, then XOR each hex-decoded line with the repeated key. The discipline never changes: decode until the string stops changing, and read the surrounding source code (`backdoor.py`) for the scheme.

3. Guided Exercises

Work through the drills on `files/capture.pcapng` (copy it to your working directory first). Every output is an exhibit: capture it, number it (B-01, B-02, and so on), and log it in Appendix B.

3.A Orientation

- 1. Run `capinfos` on the capture; record packet count, capture window, and the SHA256.
- 2. Open Statistics -> Protocol Hierarchy; list the protocols and their percentages.
- 3. Open Statistics -> Conversations; rank TCP conversations by bytes. Which dominates?
- 4. Write down the two busiest endpoints (IP:port) and what service each port suggests.

3.B Filters and details

- 1. Apply the filter `http` - how many packets survive? Then `ftp`. Then `tcp.port == 21`.
- 2. Filter for one host with `ip.addr` and combine with a protocol using `&&`.
- 3. Select one HTTP packet; expand IPv4 and TCP in the details pane and record the TTL and window size.
- 4. Right-click the User-Agent field and apply it as a filter; note the generated syntax.

3.C Streams and exports

- 1. Follow one TCP stream (right-click -> Follow -> TCP Stream); note its stream number.
- 2. Record which color is the client and which is the server; quote one line from each.
- 3. Use File -> Export Objects -> HTTP; list what is recoverable and save one object.
- 4. Compute the MD5 of the exported object and log it as evidence.

3.D tshark warm-up

- 1. `tshark -r capture.pcapng -Y ftp | head` - what do the Info column entries show?
- 2. `tshark -r capture.pcapng -Y 'ftp.request.command == "PASS"' -T fields -e ftp.arg` - list every password attempted.
- 3. `tshark -r capture.pcapng -q -z conv,tcp` - compare with the Wireshark conversations view.

3.E Your own capture (optional)

- 1. Start a capture, browse one HTTP (not HTTPS) site, stop, and save as mycapture.pcapng.
- 2. Find your own User-Agent and the GET request in a followed stream.
- 3. Reflect: what would an investigator learn about you from 60 seconds of your traffic?

4. Challenges

Challenge A is the upstream breach capture - solved together in the Deep Dive session. Challenges B and C are yours: work on copies, hash before and after, cite packet and stream numbers. Flags use `ctf{xxxx}` (Challenge A keeps the upstream flag format). Hints are staged - stop after each one and retry.

4.1 Challenge A — the breach capture (solved in the Deep Dive)

The ten questions from the lab README: the protocols present, the successful FTP password, the additional FTP loot, the use of that information, the root password, the RCE packet numbers and payload, the post-access activity, the file from root's home, the purpose of the downloaded script, and the covert channel's content. The Deep Dive walks the full solution - your deliverable is the same answers backed by YOUR exhibits.

4.2 Challenge B — office hours (files/office_hours.pcap)

- 1. Copy the capture to your working directory and record its SHA256 (capinfos).
- 2. Which two hosts are talking, and on what ports? Which traffic is background noise?
- 3. Follow the HTTP stream that contains the login attempts. How many tries fail, and what finally succeeds? Record the username and password.
- 4. One GET downloads a plain-text file. Recover it with Export Objects (or from the stream) and record its MD5.
- 5. The file contains a handover token, encoded. Decode it and write it down.
- 6. In one sentence: why was this credential capture possible at all (protocol property)?

Hints: (1) Statistics -> Conversations ranks the talkers; the quiet host is the colleague. (2) The successful POST is the one whose response is 200, not 302. (3) The token line says how it is encoded - apply that decoder once.

Flag B: `ctf{_____}`

4.3 Challenge C — the ping puzzle (files/ping_puzzle.pcap)

- 1. Copy the capture and record its SHA256. What single protocol does it contain?
- 2. Filter to ICMP echo requests only. How many distinct payload patterns do you see?

- **3.** One group of requests carries payloads that are obviously not the standard padding - quote a few and note their common marker.
- **4.** Order those payloads by sequence number (or time) and concatenate the payload text after the marker.
- **5.** Decode the concatenated string with the appropriate decoder and recover the full message and the flag.
- **6.** Some pings form a second, readable group. What do they say, and what was their purpose?

Hints: (1) icmp.type == 8 selects requests. (2) The odd payloads all start with the same punctuation character. (3) The concatenated text is one long encoded string - the alphabet it uses tells you which decoder.

Flag C: ctf{_____}

5. Analysis Questions

Questions 1-3 condense the original lab's ten questions; questions 4-12 extend the case. Cite exact packet numbers, stream numbers, ports, or filters in every answer.

1. The breach capture (files/challenge.pcapng):

- a.** Which protocols are present, and which one dominates by packet count?
- b.** Which FTP password finally succeeded, and how is that visible without reading every guess?
- c.** Which two files did the attacker retrieve over FTP, and what did they contain?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

2. Escalation and access:

- a.** What did the attacker do with the stolen database credentials?
- b.** Which packet numbers carry the RCE exploitation, and what is the decoded payload?
- c.** What is the root password, and in which stream is it exposed?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

3. Post-access and covert channel:

- a.** What was in the file read from root's home directory (quote the flag)?
- b.** What does the downloaded backdoor.py do, step by step?
- c.** Which commands traveled through the covert channel, and what was the final flag?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

4. Why does Follow TCP Stream show both directions while the packet list shows one packet at a time? What evidentiary value does the two-color layout add?**Answer:**

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

5. The attacker downloaded backdoor.py over HTTPS from GitHub. Why can you see the domain (SNI/DNS) but not the file content - and how did we still know what the script does?**Answer:**

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Explain how the FTP brute force is visible as a SHAPE (many short streams) rather than as content. Which two statistics would you set to catch it automatically?**Answer:**

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

7. The reverse shell used port 4444 and the backdoor port 5555. Why do attackers choose such ports, and what is the minimal egress rule that would have blocked both?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

8. The covert channel encoded commands as hex and XOR with a repeating key. Why is repeating-key XOR not encryption in any serious sense, and what made it 'covert' anyway?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. ICMP has no ports. How would you (a) distinguish echo requests from replies in a filter, and (b) order the covert payloads correctly if timestamps were unreliable?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. Separate IOAs from IOCs in this capture: list two of each with the stream or packet that grounds them.

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. Why must capture analysis happen on copies with hashes taken before and after? What does a mismatch prove - and what does it not prove?

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

12. Management asks for one sentence: 'what did the attacker do on our network?'. Write it - then write the one sentence you would give the SOC instead.

Answer:

Write your complete answer, citing the exact packet number, stream number, or filter used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. Reflection and Self-Assessment

Summarize your main findings, the skills that clicked, and the obstacles you hit - with how you solved them.

Reflection:

Findings, new skills, obstacles, and how you overcame them.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Orient in a capture with hierarchy, conversations, and capinfos		
Write display filters for protocols, ports, hosts, and commands		
Read and cite a TCP stream as evidence		
Recover files with Export Objects and hash them		
Spot brute force, credential leaks, and reverse shells on the wire		
Decode URL, hex, base64, and repeating-key XOR payloads		

I can	Yes	Not yet
Reassemble a covert channel from ICMP payloads		

7. Appendices

Appendix A — Tool cheat sheet

Task	Command / action
Capture metadata + hash	capinfos capture.pcapng
Protocol hierarchy (CLI)	tshark -r c.pcap -q -z io,phs
Conversations	tshark -r c.pcap -q -z conv,tcp
List FTP passwords	tshark -r c.pcap -Y 'ftp.request.command=="PASS"' -T fields -e ftp.arg
Stream ids on a port	tshark -r c.pcap -Y tcp.port==21 -T fields -e tcp.stream sort -n uniq
Dump one stream	tshark -r c.pcap -q -z follow,tcp,ascii,13
Export HTTP objects	File -> Export Objects -> HTTP (or tshark --export-objects http,outdir/)
ICMP payloads	tshark -r c.pcap -Y 'icmp.type==8' -T fields -e data
URL decode	python3 -c "import urllib.parse as u; print(u.unquote_plus(S))"
Hex + XOR decode	python3 -c "print(bytes(a^b for a,b in zip(bytes.fromhex(H), (K*99)[:len(bytes.fromhex(H))])).decode())"
Live capture (CLI)	sudo tcpdump -i eth0 -w capture.pcap

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

What good evidence looks like — model exhibit:

```
[B-04] 2026-10-19 09:12 Wireshark, office_hours.pcap
Follow TCP Stream #3 (filter: tcp.stream==3)
    PASS batman
    230 Login successful.
-> captured as evidence/B-04_ftp_stream3.png
```

- The stream number, the filter to re-find it, and both directions are visible.
- The tool and moment of capture are recorded; the filename says what it shows.
- One exhibit supports one finding.
- Bad evidence: a paraphrased password with no stream or packet number.

No.	Time	Activity / Filter	Source file / Tool	Notes
B-01				
B-02				
B-03				
B-04				
B-05				
B-06				
B-07				
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per capture file: hash when received (start) and after analysis (end). Identical values prove faithful handling.

No	Capture file	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No
5					Yes / No

Appendix D — Glossary

Term	Meaning in this lab
packet capture (pcap/pcapng)	a recording of packets as they crossed the wire; pcapng is the modern format
conversation / stream	one (IP:port <-> IP:port) exchange; Wireshark numbers TCP streams
well-known ports	21 FTP, 22 SSH, 80 HTTP, 443 HTTPS, 3306 MySQL - the service map

Term	Meaning in this lab
display filter	Wireshark's query language (http, tcp.port==5555, ip.addr==x)
protocol hierarchy	statistics view: protocols and their share of the capture
Follow TCP Stream	reassembles one conversation, both directions, color-coded
Export Objects	recovers files transferred over HTTP/SMB/FTP-DATA
capinfos	capture metadata: window, counts, and hashes
brute force	many credential guesses; on the wire: many short streams + failure replies
reverse shell	victim connects out to the attacker's listener (here port 4444)
covert channel	data smuggled through traffic that looks protocol-normal (e.g., odd ICMP payloads)
repeating-key XOR	bitwise XOR against a cycled key; trivially reversible once the key is known
SNI / DNS leakage	the hostname remains visible even in TLS traffic
IOA / IOC	indicator of attack (attempted) vs indicator of compromise (successful)
egress filtering	outbound firewall rules - the minimal control that kills C2 ports
working copy	the capture copy you analyze; originals stay sealed and hashed

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____